You said:

codigo(2).c
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 30s

ChatGPT said:

Análise de segurança do arquivo `codigo(2).c`

MALWARE — ALTA CONFIANÇA

Veredito: o arquivo é malicioso.

O código implementa um **backdoor para Linux com acesso remoto não autorizado (Remote Access Trojan — RAT)**. Sua finalidade é permitir que um operador remoto controle o sistema comprometido por meio de um shell (`/bin/sh`), utilizando pacotes de rede especialmente construídos para ativar a funcionalidade.

Não se trata apenas de um programa com vulnerabilidades ou comportamento suspeito: existem funcionalidades explícitas de acesso remoto, ocultação de processo, comunicação criptografada e manipulação de regras de firewall.

1. Identificação e características iniciais

O arquivo é um código-fonte escrito em C, com aproximadamente 879 linhas.

Ele utiliza bibliotecas relacionadas a:

- Comunicação TCP e UDP.
- Criação de sockets RAW.
- Manipulação de processos e sinais.
- Criação de terminais pseudo-TTY.
- Controle de interfaces de rede.
- Criptografia RC4.

A presença dessas bibliotecas, isoladamente, não comprova atividade maliciosa. Entretanto, sua combinação com as funcionalidades encontradas posteriormente é bastante reveladora.

2. Análise das funcionalidades suspeitas

2.1. Comunicação criptografada com RC4

O programa implementa o algoritmo RC4 por meio das funções:

- `rc4_init()` : inicializa o estado da criptografia utilizando uma chave.
- `rc4()` : realiza a criptografia e descriptografia dos dados.
- `cwrite()` : criptografa os dados antes de enviá-los pela rede.
- `cread()` : descriptografa os dados recebidos.

Essas funções são utilizadas para proteger a comunicação entre o invasor e o sistema comprometido.

Referência: linhas 95–118, 139–212. codigo.c codigo.c

Avaliação: comportamento compatível com um backdoor que procura dificultar a inspeção do tráfego de controle.

2.2. Ocultação da presença do malware

Uma funcionalidade particularmente suspeita está na função `set_proc_name()`.

Ela:

- Modifica o conteúdo de `argv[0]`.
- Sobrescreve o nome apresentado pelo processo.
- Utiliza `prctl(PR_SET_NAME, ...)` para alterar o nome do processo no sistema operacional.

Além disso, o código contém uma lista de nomes associados a processos legítimos do Linux, como:

- `udevd`
- `mingetty`
- `console-kit-daemon`
- `dbus-daemon`
- `auditd`

O programa escolhe aleatoriamente um desses nomes para disfarçar sua identidade.

Referência: linhas 370–404 e 817–831. codigo.c codigo.c

Há também uma string codificada em hexadecimal que, após decodificação, corresponde a:

```
/usr/libexec/postfix/master
```

Esse nome é utilizado para disfarçar outro processo criado pelo malware.

Referência: linhas 545–560. codigo.c

Avaliação: forte evidência de técnicas de evasão e ocultação de processos.

2.3. Cópia e execução em diretório temporário

A função `to_open()` monta um comando de sistema a partir de uma string codificada em hexadecimal.

Após decodificação, o comando executa essencialmente as seguintes operações:

1. Remove uma eventual cópia anterior de `/dev/shm/kdmtmpflush`.
2. Copia o próprio executável para esse diretório.
3. Concede permissão de execução ao arquivo.
4. Executa a cópia com o argumento `--init`.
5. Remove a cópia quando o processo executado termina.

Referência: linhas 406–429 e 851–854. codigo.c codigo.c

O diretório `/dev/shm` normalmente utiliza memória compartilhada, o que permite armazenar arquivos temporários sem mantê-los necessariamente em um sistema de arquivos persistente.

Avaliação: mecanismo de execução temporária que pode dificultar a localização do arquivo malicioso.

2.4. Captura de pacotes de rede

Uma das evidências mais importantes aparece na função `packet_loop()`.

O programa cria um socket RAW:

```
socket(PF_PACKET, SOCK_RAW, htons(ETH_P_IP))
```

Em seguida, instala um filtro BPF utilizando:

```
setsockopt(sock, SOL_SOCKET, SO_ATTACH_FILTER,
           &filter, sizeof(filter))
```

O filtro verifica características específicas dos pacotes, incluindo o protocolo de rede e o marcador hexadecimal `0x7255`.

Posteriormente, o programa recebe os pacotes com `recvfrom()` e interpreta seus conteúdos como estruturas TCP, UDP, ICMP e `magic_packet`.

Referência: linhas 447–535 e 497–509. codigo.c

A estrutura `magic_packet` contém:

Campo	Finalidade
`flag`	Identificador do pacote
`ip`	Endereço IP de destino
`port`	Porta de comunicação
`pass`	Senha de autenticação

Referência: linhas 84–88. codigo.c

Avaliação: o programa implementa um mecanismo de ativação remota por pacotes especialmente construídos, compatível com a técnica conhecida como *magic packet* ou *packet-triggered backdoor*.

2.5. Autenticação por senhas predefinidas

O código possui duas senhas armazenadas diretamente no programa.

Após decodificação dos valores hexadecimais:

Variável	Senha
`hash`	`justforfun`
`hash2`	`socket`

A função `logon()` compara a senha recebida no pacote com esses valores.

Referência: linhas 432–442 e 817–862. codigo.c codigo.c

A partir dessa comparação, são estabelecidos três comportamentos:

- **Senha** `justforfun` **:** tenta estabelecer uma conexão TCP com o endereço e a porta fornecidos no pacote.
- **Senha** `socket` **:** prepara uma conexão de entrada para permitir o acesso remoto.
- **Senha inválida:** envia um pacote UDP contendo o caractere `1` para o endereço e a porta informados.

Referência: linhas 562–575. codigo.c

Avaliação: autenticação por senha embutida em um protocolo de controle remoto clandestino.

2.6. Criação de acesso remoto por shell

Esta é a evidência mais conclusiva da finalidade maliciosa.

O programa implementa duas modalidades principais de acesso.

A. Conexão de saída (Reverse Shell)

Quando recebe a senha `justforfun` , executa:

```
scli = try_link(bip, mp->port);

if (scli > 0)
    shell(scli, NULL, NULL);
```

A função `try_link()` cria um socket TCP e tenta estabelecer uma conexão com o endereço IP e a porta recebidos.

Caso a conexão seja estabelecida, a função `shell()` disponibiliza um terminal remoto.

Referência: linhas 325–345 e 568–571. codigo.c codigo.c

B. Conexão de entrada (Bind Shell com redirecionamento)

Quando recebe a senha `socket` , o malware executa `getshell()` .

Essa função:

1. Procura uma porta TCP disponível entre 42391 e 43390.
2. Cria um socket de escuta.

3. Modifica regras do firewall utilizando `iptables`.
4. Configura redirecionamento de conexões com NAT.
5. Aguarda uma conexão externa.
6. Quando a conexão é estabelecida, disponibiliza um shell remoto.

O código manipula especificamente as cadeias `PREROUTING` e `INPUT` do firewall.

Referência: linhas 585–612 e 631–682. codigo.c codigo.c

Avaliação: capacidade explícita de fornecer acesso remoto não autorizado ao sistema.

2.7. Execução de comandos no sistema comprometido

A função `shell()` executa:

```
execve(sh, argvv, envp);
```

A variável `sh` contém:

```
/bin/sh
```

O programa utiliza um pseudo-terminal (PTY) para oferecer uma sessão interativa, redirecionando entrada e saída entre o socket de rede e o terminal.

Também configura:

- `HISTFILE=/dev/null`
- `MYSQL_HISTFILE=/dev/null`
- Um diretório HOME temporário.
- Um nome de processo disfarçado como `qmgr`.

Essas configurações ajudam a ocultar atividades realizadas durante a sessão remota.

Referência: linhas 686–761 e 763–814. codigo.c codigo.c

Avaliação: comprovação de que o programa possibilita executar comandos arbitrários por meio de um terminal remoto.

3. Execução com privilégios e comportamento de ocultação

O programa verifica se está sendo executado como usuário root:

```
if (getuid() != 0) {
    return 0;
}
```

Portanto, suas funcionalidades principais dependem de privilégios administrativos.

Ele também:

- Cria o arquivo `/var/run/haldrun.pid`.
- Impede outra inicialização quando esse arquivo já existe.
- Executa operações de daemonização utilizando `fork()` e `setsid()`.
- Instala tratamento de sinais para remover o arquivo PID quando termina normalmente.
- Altera os horários de acesso e modificação do arquivo executado, utilizando `utimes()`.

Referência: linhas 220–231 e 843–878. codigo.c codigo.c

Observação: não identifiquei neste código um mecanismo explícito de inicialização automática após reinicialização, como uma entrada em cron ou serviço systemd. O arquivo PID e a daemonização, por si só, não comprovam persistência após reboot.

4. Indicadores de comprometimento (IOCs)

A análise identificou os seguintes indicadores que podem auxiliar na investigação de um sistema comprometido.

Indicador	Valor
Arquivo temporário	`/dev/shm/kdmtmpflush`
Arquivo PID	`/var/run/haldrun.pid`
Processo disfarçado	`/usr/libexec/postfix/master`
Faixa de portas TCP	42391–43390
Marcador de ativação	`0x7255`
Senha 1	`justforfun`
Senha 2	`socket`
Criptografia	RC4

Hash SHA-256 do arquivo analisado:

```
231756944eefc3a2dd7c2ab62d8cc05d005a1deae2d7c5d1e17db18c9885b52d
```

Não foram identificados endereços IP ou domínios de comando e controle fixos no código analisado. Os endereços e portas de comunicação são fornecidos dinamicamente pelos pacotes recebidos.

5. Resultado final da análise

CLASSIFICAÇÃO: MALWARE

Backdoor / RAT para Linux

Categoria principal: Backdoor de acesso remoto.

Funcionalidades secundárias identificadas:

- Reverse Shell.
- Bind Shell.
- Ocultação de processos.
- Comunicação criptografada.
- Ativação por pacotes de rede especiais.
- Manipulação de firewall.
- Execução com privilégios de root.

Justificativa da classificação:

O arquivo apresenta evidências inequívocas de comportamento malicioso. Sua finalidade é permitir que um operador remoto obtenha acesso interativo ao sistema Linux, execute comandos arbitrários, utilize conexões de entrada ou saída e dificulte a identificação do processo responsável.

A combinação de ativação clandestina por pacotes de rede, senhas embutidas, comunicação RC4, disfarce de processos e execução de shell remoto elimina uma interpretação razoável de que se trata de um programa administrativo benigno.

Conclusão definitiva: `codigo(2).c` **é um malware da categoria Backdoor/RAT para Linux.**

A análise foi realizada estaticamente, sem compilar ou executar o código. Caso esse programa tenha sido executado em um servidor, recomendo isolar o equipamento da rede e verificar especialmente os indicadores de comprometimento apresentados acima.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Backdoor (porta dos fundos)**, com funcionalidades de **RAT (Remote Access Trojan)** para Linux.

Ele permite que um invasor obtenha acesso remoto ao sistema e execute comandos arbitrários por meio de um shell remoto.